

CLASS PRACTICE/ASSIGNMENT

Decision Tree Assignment: Regression and Classification

Objective

Implement Decision Tree Regression and Decision Tree Classification using sklearn, evaluate both models, apply pruning/hyperparameter tuning, and write clear interpretation from EDA, model metrics, and tree behavior.

Step-wise implementation

1. Import both datasets: one for regression and one for classification.
2. Identify the target columns:
 - Regression target: Collection
 - Classification target: Start_Tech_Oscar
3. Perform EDA for both datasets:
 - shape, column names, data types
 - missing values
 - duplicate values
 - target distribution
 - numeric summary statistics
 - correlation with target
 - important visualizations
4. Separate features and target:
 - X_reg, y_reg for regression
 - X_clf, y_clf for classification
5. Handle preprocessing:
 - remove/handle missing values
 - encode categorical columns
 - check whether any column causes leakage
6. Split both datasets into train and test data using 80:20 ratio.
7. Train baseline models:

- DecisionTreeRegressor
- DecisionTreeClassifier

8. Evaluate initial unpruned models:

- Regression: MAE, MSE, RMSE, R2
- Classification: Accuracy, Precision, Recall, F1-score, Confusion Matrix

9. Visualize both trees and explain:

- root node
- important split conditions
- leaf nodes
- samples per leaf
- overfitting signs

10. Apply pruning/hyperparameter tuning:

- max_depth
- min_samples_split
- min_samples_leaf
- max_leaf_nodes
- ccp_alpha

11. Re-train the pruned models and compare:

- before pruning vs after pruning
- train score vs test score
- tree depth and number of leaves

12. Write final insights:

- Which features are important?
- Did pruning improve generalization?
- Is the model overfitting or underfitting?
- Which model is better for explanation?
- Which model is better for prediction?

Submission expectations

Students must submit one Jupyter Notebook containing:

- clean section headings
- code cells
- markdown explanation after every result
- visualizations
- model comparison table
- final conclusion for both regression and classification

Use this **10-line starter code only** as a skeleton;

students should expand it with EDA, plots, metrics, pruning, and explanation.

...

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score,
accuracy_score, classification_report, confusion_matrix

reg_df, clf_df = pd.read_csv("Movie_regression.csv"),
pd.read_csv("Movie_classification.csv")

Xr, yr = pd.get_dummies(reg_df.drop("Collection", axis=1), drop_first=True),
reg_df["Collection"]

Xc, yc = pd.get_dummies(clf_df.drop("Start_Tech_Oscar", axis=1), drop_first=True),
clf_df["Start_Tech_Oscar"]

Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(Xr, yr, test_size=0.2, random_state=42)

Xc_tr, Xc_te, yc_tr, yc_te = train_test_split(Xc, yc, test_size=0.2, random_state=42,
stratify=yc)

reg_model, clf_model = DecisionTreeRegressor(random_state=42),
DecisionTreeClassifier(random_state=42)

reg_model.fit(Xr_tr, yr_tr); clf_model.fit(Xc_tr, yc_tr)

reg_pred, clf_pred = reg_model.predict(Xr_te), clf_model.predict(Xc_te)
```

...

sklearn references for students

- DecisionTreeRegressor: official sklearn regressor for continuous-value prediction, with pruning support through ccp_alpha. ([scikit-learn](#))
- DecisionTreeClassifier: official sklearn classifier for class-label prediction. ([scikit-learn](#))
- train_test_split: used to split arrays/dataframes into random train and test subsets. ([scikit-learn](#))
- Cost-complexity pruning: sklearn explains that larger ccp_alpha values prune more nodes. ([scikit-learn](#))
- Regression metrics: mean_squared_error, mean_absolute_error, and r2_score. ([scikit-learn](#))
- Classification metrics: accuracy_score, classification_report, confusion_matrix, and f1_score. ([scikit-learn](#))
- GridSearchCV: use it for systematic hyperparameter tuning with cross-validation. ([scikit-learn](#))